Laboratoire 2
Conception du serveur d'application et du client

1 Introduction

Au cours de la session, les laboratoires auront comme sujet le développement d'une application de gestion de données, à l'image des projets qui se font en entreprise. Chacun des quatre (4) laboratoires du cours portera sur un ou plusieurs concepts vus en classe, et sera effectué en équipe de **trois (3) à quatre (4)** étudiants.

L'objectif de ce second laboratoire est de vous familiariser avec la conception d'une application Web à trois niveaux (*three-tier application*) dont les données transactionnelles sont stockées dans une BD relationnelle. En particulier, ce laboratoire portera sur l'utilisation du framework de mappage objet/relationnel (ORM) pour assurer la persistance transparente des données.

Le travail à réaliser dans le cadre de ce laboratoire se base sur celui accompli lors du laboratoire précédent. Ainsi, l'application Web à développer devra utiliser le schéma relationnel conçu au laboratoire précédent, de même que les données qui y ont été insérées.

2 Travail à réaliser

Tâche 1 : Modélisation des classes de l'application en Java

En se basant sur le diagramme de classe UML fait lors du premier laboratoire, vous devrez tout d'abord coder toutes les classes métier nécessaires pour traiter **SEULEMENT les cas d'utilisation 2, 3 et 4** du document de mise en situation, soit :

- Cas 2 : connexion d'un utilisateur au système ;
- Cas 3 : consultation interactive des films ;
- Cas 4 : location de films.

Vous devrez également coder la ou les classes nécessaires pour interfacer ces classes métiers à l'aide du patron de façade. Toute requête complexe au système (ex : recherche de films, insertion d'une location, etc.), provenant de la couche de contrôle/présentation, devrait se faire à travers un objet de la façade, et non directement sur les objets métier.

Consigne 1 : Vous pouvez, si nécessaire, apporter des modifications à votre diagramme de classe ou votre schéma relationnel. Ces modifications peuvent faciliter la mappage objet-relationnel de la Tâche 2.

Consigne 2 : Bien que vous n'êtes pas tenu de mettre des commentaires pour chaque élément de votre code, vous serez pénalisé si celui-ci n'est pas suffisamment compréhensible. En particulier, choisissez des noms appropriés et significatifs pour vos classes, attributs et méthodes.

Tâche 2 : Mappage des objets à la BD avec Hibernate ou autre ORM

Pour cette seconde tâche, vous devrez définir le mappage de vos objets aux tables de la BD en utilisant le framework Hibernate ou équivalent. Pour Hibernate:

Dans un premier temps, vous devrez modifier le fichier de configuration `hibernate.cfg.xml` se trouvant dans le répertoire `src/main/resources`, afin de mettre les bons paramètres de connexion (`username`, `password` et `default_schema`) :

```
<hibernate-configuration>
 <session-factory name="Hibernate">
  <property name="hibernate.connection.driver_class">
     oracle.jdbc.driver.OracleDriver</property>
  <property name="hibernate.connection.password">
     leMotDePasse</property>
  <property name="hibernate.connection.url">
     jdbc:oracle:thin:@big-data-3.logti.etsmtl.ca:1521:LOG660</property>
  <property name="hibernate.connection.username">
     equipeXX</property>
  <property name="hibernate.dialect">
     org.hibernate.dialect.Oracle10gDialect</property>
  <property name="hibernate.default_schema">
     EQUIPEXX</property>
  <property name="hibernate.transaction.auto_close_session">
     false</property>
 </session-factory>
</hibernate-configuration>
```

À noter : la valeur du paramètre `default_schema` doit être en lettres majuscules.

Ensuite, vous devrez créer, pour chacune des classes métier nécessaires pour traiter **SEULEMENT les cas d'utilisation 2, 3 et 4**, un fichier de mappage (`hibernate-mapping`) définissant :

- Le mappage de la classe vers une table ;
- Le mappage de l'identifiant de la classe vers une clé primaire, et le mecanisme de generation si la clé est artificielle ;
- Le mappage des propriétés de la classe vers des colonnes de sa table ;
- Le mappage des relations d'association (ex : 1-1, 1-plusieurs, plusieurs-1, etc.);

- Le mappage des relations de spécialisation (ex : une table par hiérarchie, par sous-classe, etc.)

Voici un exemple de fichier de mappage `Film.hbm.xml` :

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD//EN"
    "http://hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="Film" table="FILM">
        <id name="id" column="ID_FILM"/>
            <generator class="native"/>
        </id>
        <property name="titre" column="TITRE"/>
        <property name="annee" column="ANNEE_SORTIE"/>
        ...
        <set name="lesActeurs" inverse="true" cascade="all">
            <key column="ID_FILM"/>
            <one-to-many class="Personne"/>
        </set>
    </class>
</hibernate-mapping>
```

Consigne 1 : Basez-vous sur les acétates présentées en classes pour la définition des fichiers de mappage. Vous pouvez également vous référer à la document en-ligne de Hibernate disponible ici.

Consigne 2 : Utilisez le mode `native` pour la stratégie de génération des clés artificielles.

Consigne 3 : Si vous préférez, les annotations Hibernate peuvent être employées pour faire le mappage, au lieu des fichiers XML.

Tâche 3 : Implémentation du client mince

Dans cette troisième tâche, vous aurez à développer la couche de présentation de votre application sous la forme d'un client mince. Ce client doit pouvoir traiter **SEULEMENT les cas d'utilisation 2, 3 et 4**.

Pour le développement de votre client, vous avez deux options :

1. Client Web à l'aide de Servlet Java (ou technologie équivalente au choix) ;
2. Interface GUI au choix (ex : Swing).

Pour la première option, référez-vous au document "*Application Web et Servlet Java*" disponible avec cet énoncé pour des instructions détaillées. Le squelette d'un client Web vous sera également fourni pour faciliter votre travail. **Note :** bien qu'il ne soit pas obligatoire d'implémenter un client Web, cela cadre mieux dans la philosophie d'une application à trois couches.

À noter : toute opération transactionnelle de l'application doit être faite à l'intérieur d'une session Hibernate. Par exemple :

```
Session uneSession =HibernateUtil.getSessionFactory().openSession();
uneSession.beginTransaction();

List lesClients = uneSession.createQuery(
 "FROM Client c WHERE c.courriel = 'jean.bon@etsmtl.ca' ").list();
Client unClient = (Client) lesClients.next();

List lesCopies = uneSession.createQuery(
 "FROM Copie c WHERE c.titre = 'La revanche du Jean Bon' AND
  c.statut = 'disponible' ").list();
Copie uneCopie = (Copie) lesCopies.next();

Client.getLocations().add(uneCopie);
uneCopie.setEmprunteur(unClient);
uneCopie.setStatut("louee");

uneSession.getTransaction().commit();
uneSession.close();
```

Consigne 1 : Ne passez pas trop de temps sur le côté esthétique de l'application, car celui-ci ne sera pas évalué.

Consigne 2 : Assurez de bien tester toutes les fonctionnalités de votre application, y compris les utilisations non-prévues et autres cas d'erreurs.

Consigne 3 : Les mêmes règles d'affaires (contraintes) que pour le laboratoire 1 s'appliquent au laboratoire 2. Par exemple, un client ne peut toujours pas louer plus de films que le permet son forfait. Ses règles peuvent être validées soit au niveau de la BD (ex : les TRIGGER développées au laboratoire 1) ou au niveau de l'application.

Tâche 4 : Rédaction du rapport

Enfin, la dernière tâche du laboratoire sera de rédiger un rapport décrivant votre travail et justifiant vos décisions de conception.

Note : utilisez le gabarit prévu à cet effet.

3 Consignes de remise

- Avant la date limite disponible sur le calendrier du cours, vous devrez remettre sur Moodle les deux éléments suivants :

  Le gabarit de rapport complété ;

  1. Tous les fichiers de configuration et de mappage ;
  2. Le code source de votre application (serveur et client mince).

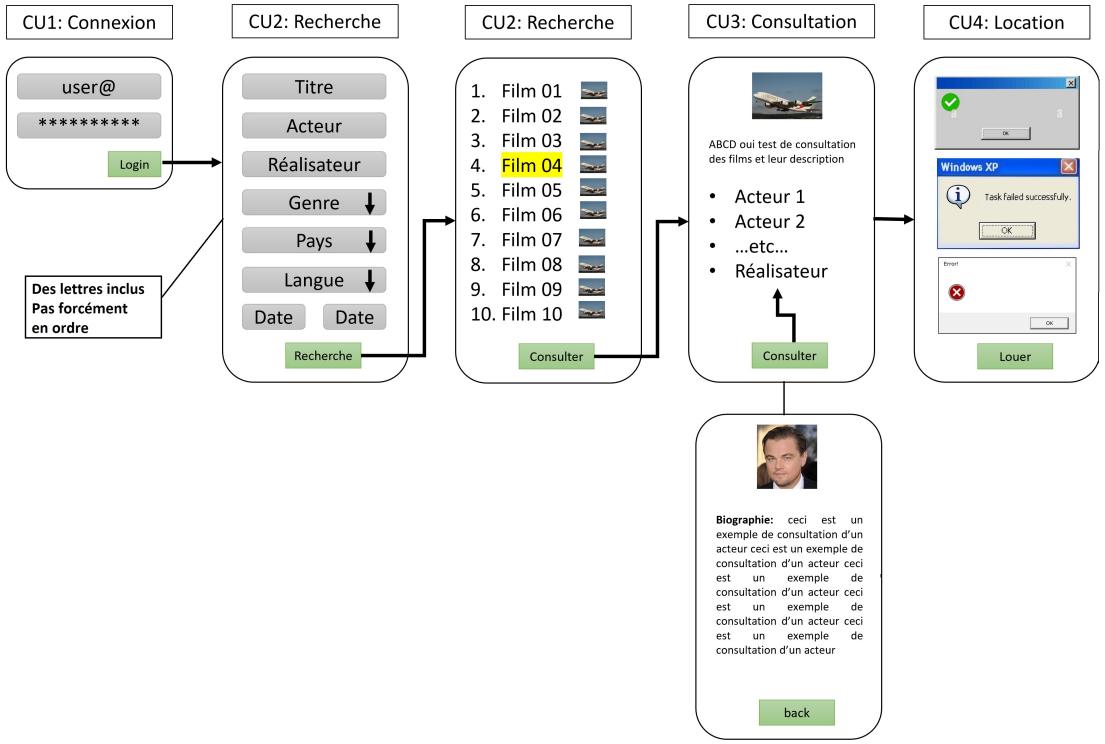
Note : Tous les fichiers de remise doivent être dans un répertoire compressé ayant comme nom `log660-labo2-equipeX`, où `X` est le numéro de votre équipe. Consultez le site du cours pour avoir plus de détails sur les consignes de remise.

4 Barème de correction

L'évaluation du laboratoire sera faite sur un total de 100 points, distribués selon le fichier excel: Grille de correction lab 2

5 Pénalités et retard

Un travail remis en retard recevra automatiquement la note 0. Une pénalité allant jusqu'à 10% pourra également être appliquée à tout travail ne respectant pas rigoureusement les directives de remise.



- Exemple de UI avec les 4 CUs -